



BITS Pilani

Modern Database Systems

Prof. Ashish Narang
CSIS



BITS Pilani



<SSZG507, Modern Database Systems>

Lecture No. 1

Course Contents



- ❖ Module 1: Foundations of Structured Data Management
- ❖ Module 2: Distributed Database Architecture and Consistency Models
- ❖ Module 3: NoSQL Data Stores
- ❖ Module 4: Cloud Databases and NewSQL Systems
- ❖ Module 5: Data Warehouses, Data Lakes and Data Lakehouse
- ❖ Module 6: Specialized Databases for Modern Applications
- ❖ Module 7: Emerging Trends in Data Systems

- **Ramez Elmasri & Shamkant B. Navathe**, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2017.
- **Martin Kleppmann**, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st Edition, O'Reilly Media, 2017.
- **Paulraj Ponniah**, *Data Warehousing Fundamentals for IT Professionals*, 2nd Edition, John Wiley & Sons, 2010.

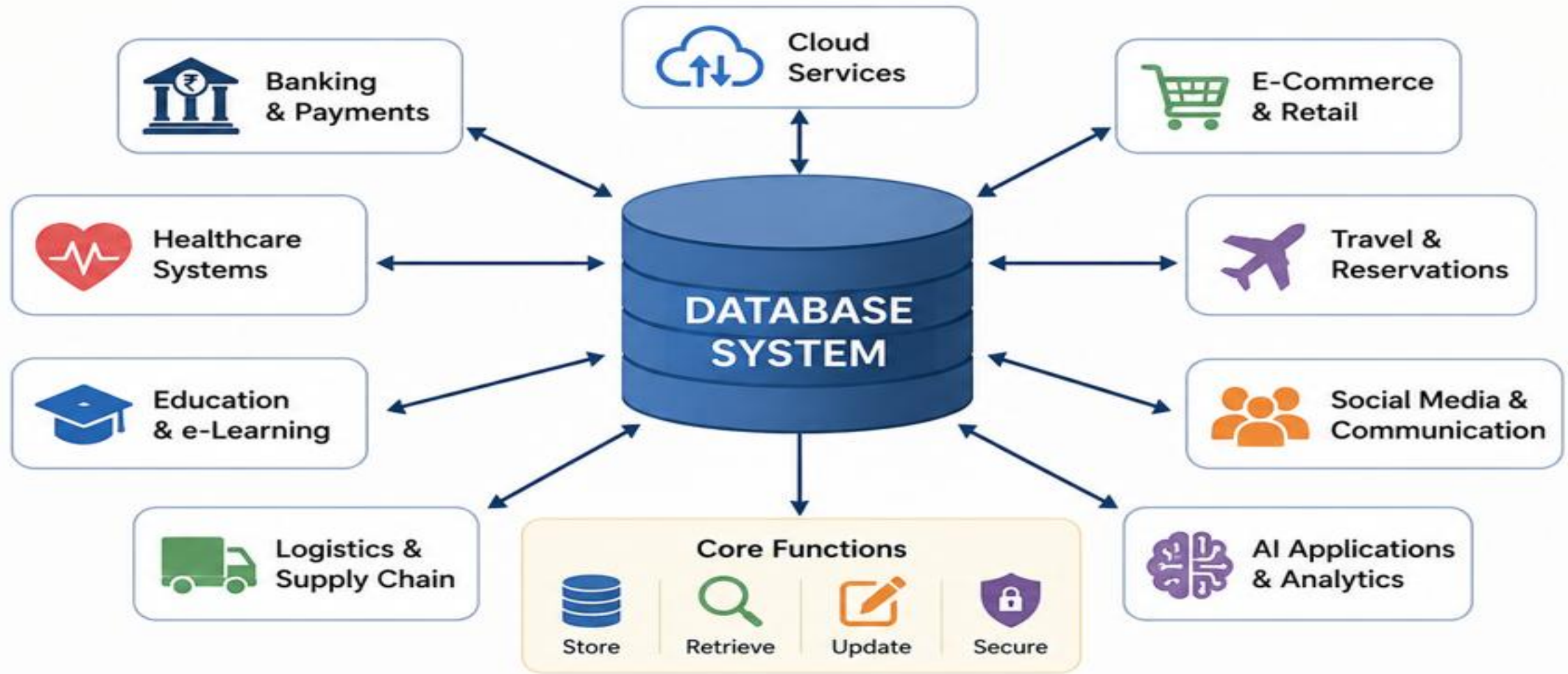
Evaluation Components



Evaluation Component	Name	Type	Weightage	Duration	Day, Date, Session, Time
	(Quiz, Lab, Project, Mid term exam, End semester exam, etc)	(Open book, Closed book, Online, etc.)			
EC – 1	Quiz 1	Online	5%	10-15 minutes	To be announced
	Quiz 2	Online	5%	10-15 minutes	To be announced
	Assignment-1	Take Home	10%	To be decided	To be announced
	Assignment-2	Take Home	10%	To be decided	To be announced
EC – 2	Mid Semester Test	Closed Book	30%	2 hours	As announced by WILP Exam Team
EC – 3	Comprehensive Examination	Open book	40%	2.5 hours	As announced by WILP Exam Team

- Introduction to Data and Databases
- Limitations of Traditional File Systems
- Types of Data and Their Characteristics
- The Relational Data Model
- Data Modelling using ER Diagrams
- Basics Database Normalization

Why Database Systems



Data

- Data represents raw facts about people, objects, events, or transactions.
- Examples: Student Name, Employee ID, Course Code, Marks, City.

Database

- A database is an organized collection of logically related data.
- It is designed to efficiently store, retrieve, update, and manage information for a specific application or organization.
- Examples: University Database, Banking Database, Hospital Database.

Database Management System (DBMS)

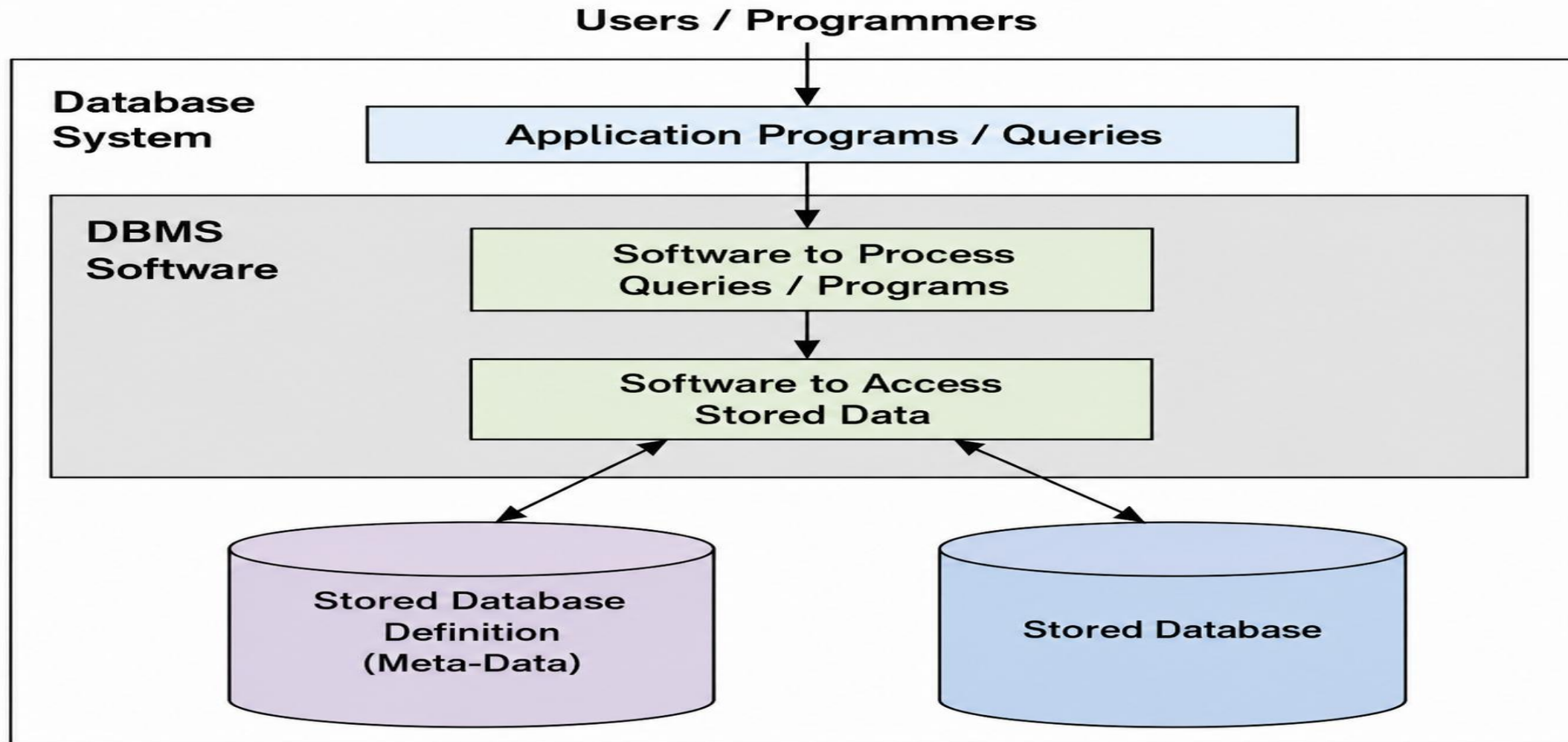


A **Database Management System (DBMS)** is software that enables users to create, store, organize, manage, and retrieve data efficiently from a database.

A DBMS facilitates:

- **Defining the database**
Specify the database structure, tables, relationships, and constraints.
- **Constructing the database**
Create database objects and populate them with data.
- **Manipulating the database**
Insert, update, delete, and retrieve data.
Support multiple users and applications simultaneously.
- **Examples: MySQL, PostgreSQL, MongoDB, CockroachDB**

Database System Environment



Traditional File Systems



- **Before Database Management Systems**, Organizations traditionally stored data in separate operating system files.
- If we take the example of savings bank enterprise, information about customers and savings accounts etc. need to be stored.
- One way to keep the information on computers is to store in the files provided by operating systems (OS).
- **Example: A Banking System**
 - Customer records → customers.txt
 - Account details → accounts.txt
 - Transactions → transactions.txt
 - Loans → loans.txt

Challenges with File Based Storage



- **Data Redundancy** – The same data may be stored in multiple files, leading to unnecessary duplication.
- **Data Inconsistency** – Updates made in one file may not be reflected in others.
- **Difficult Data Access** – Retrieving information often requires writing or modifying application programs.
- **Limited Data Sharing** – Data stored in separate files is difficult to share across applications.
- **Poor Security** – File systems provide only basic access control mechanisms.
- **No Centralized Backup and Recovery** – Recovering data after failures is difficult and time-consuming.
- **Integrity Constraints are Difficult to Enforce** – Ensuring valid and consistent data across files is challenging.

Advantages of DBMS Approach



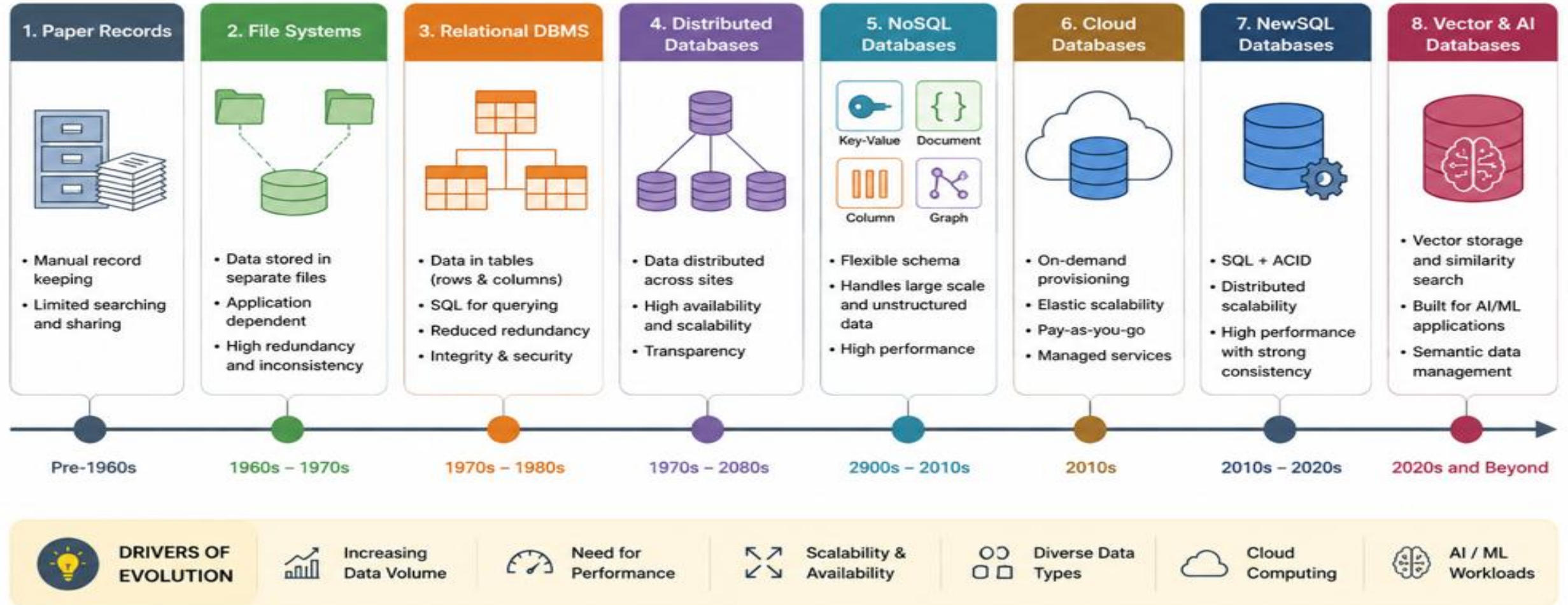
- **Controlled Data Redundancy** – Minimizes duplicate data, reducing storage requirements and improving data consistency.
- **Efficient Data Access** – Enables fast retrieval, insertion, updating, and deletion of data using query languages.
- **Data Integrity** – Ensures data accuracy and consistency by enforcing integrity constraints.
- **Enhanced Security** – Provides authentication, authorization, and access control to protect sensitive information.
- **Concurrency Control** – Allows multiple users to access and modify the database simultaneously while maintaining consistency.
- **Backup and Recovery** – Protects data from hardware or software failures through backup and recovery mechanisms.
- **Reduced Application Development Time** – Provides built-in services such as data management, security, and transaction processing, reducing development effort.

Evolution of Databases

innovate

achieve

lead



Types of Data

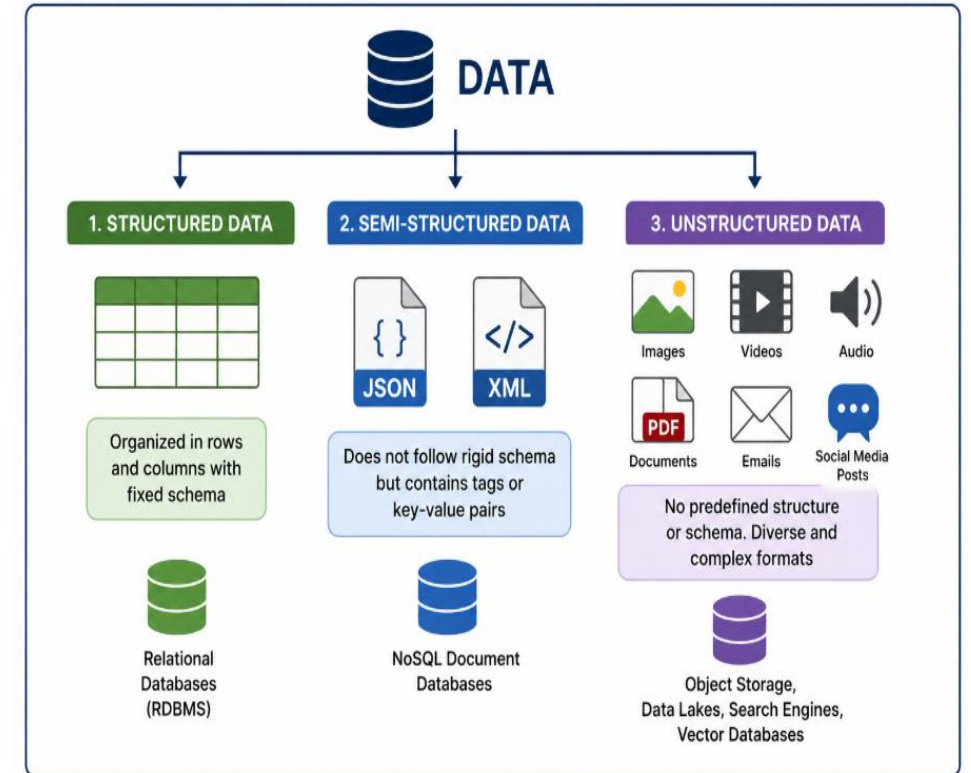


Data can be classified based on how it is organized and represented. Different types of data require different storage and management techniques.

1. Structured Data

2. Semi-Structured Data

3. Unstructured Data



Structured data is data that is organized according to a predefined schema, making it easy to store, retrieve, and process.

Characteristics

- Organized into **rows and columns**
- Follows a **fixed schema**
- Each attribute has a defined data type
- Easily queried using **SQL**
- Ensures consistency and integrity

Student_id	Name	Age
A120	Raju	21
A134	Kiran	19
C110	John	22

Semi-Structured Data



Semi-structured data does not follow a rigid tabular schema but contains tags, keys, or metadata that provide structure and organization.

Characteristics

- Does not follow a fixed schema
- Self-describing using tags or key–value pairs
- Flexible and easily extensible
- Suitable for exchanging and integrating data
- Commonly stored in JSON, XML, and YAML formats

```
{  
  "student_id": 101,  
  "name": "Aditi",  
  "programme": "M.Tech",  
  "semester": 2,  
  "cgpa": 8.9,  
  "skills": [  
    "Python",  
    "SQL",  
    "Data Analytics"  
  ]  
}
```

Unstructured Data



Unstructured data is information that does not follow a predefined schema or organized format, making it difficult to store and analyze using traditional relational databases.

Characteristics

- No predefined schema or fixed structure
- Contains text, images, audio, video, and documents
- Difficult to query using traditional SQL
- Requires specialized storage and processing techniques
- Forms the majority of data generated today
- Examples include documents, images, photographs, Audio Files, Video Files



Structured vs Semi-Structured vs Unstructured



Feature	Structured	Semi-Structured	Unstructured
Schema	Fixed	Flexible	No Fixed Schema
Storage	Tables	JSON, XML	Files, Objects
Query	SQL	JSONPath, XPath	AI/Search Engines
Examples	Student Records	API Responses	Images, Videos, PDFs
Databases	MySQL, PostgreSQL	MongoDB	Object Storage, Elasticsearch, Vector Databases

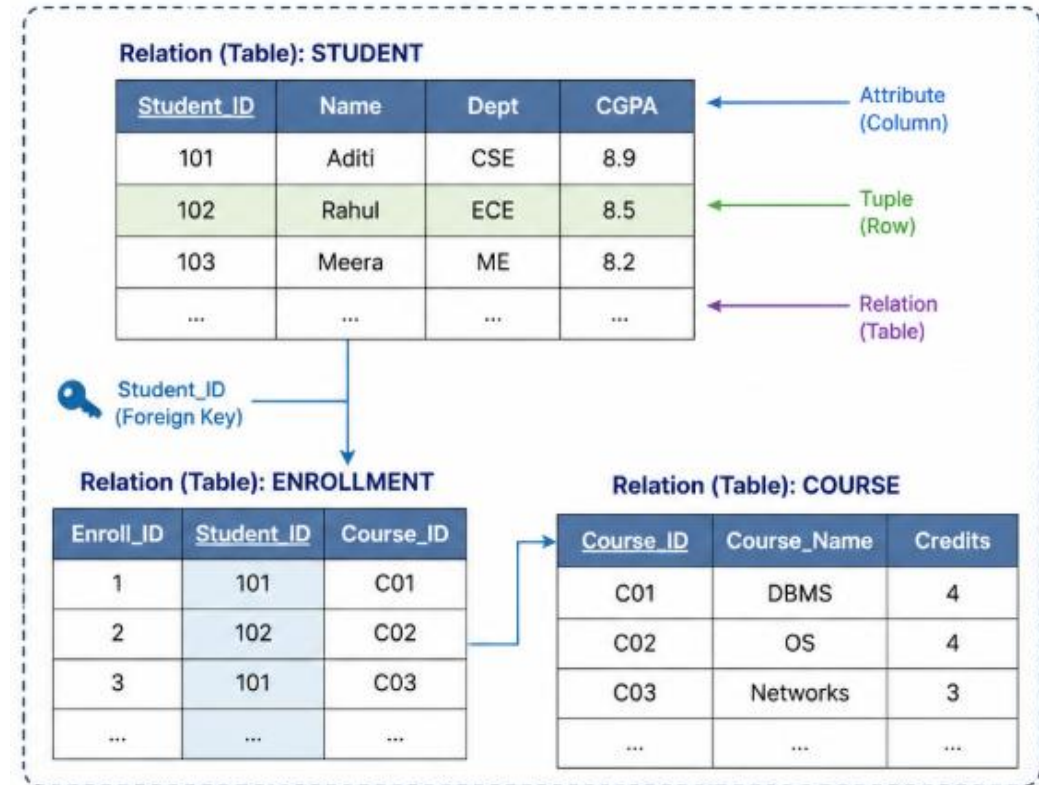
Relational Model



The Relational Model organizes data into tables (relations) and represents relationships among data using keys.

Key Characteristics

- Represents data as **relations (tables)**.
- Each relation consists of **rows (tuples)** and **columns (attributes)**.
- Each relation has a **unique name**.
- Relationships between relations are established using **keys**.
- Supports **high data integrity** through constraints.
- Uses **SQL** for data definition and manipulation.



Relational Model Constraints



- A relational database typically consists of multiple related tables containing interrelated data.
- To ensure the **accuracy, consistency, and validity** of the data, the relational model defines a set of constraints.
- These constraints help maintain **data integrity** and prevent invalid or inconsistent data from being stored in the database.
- Relational model constraints can be broadly classified into:
 - **Schema-Based Constraints**
 - Domain Constraints
 - Key Constraints
 - Entity Integrity Constraints
 - Referential Integrity Constraints
 - **Application-Based (Semantic) Constraints**

Domain Constraint



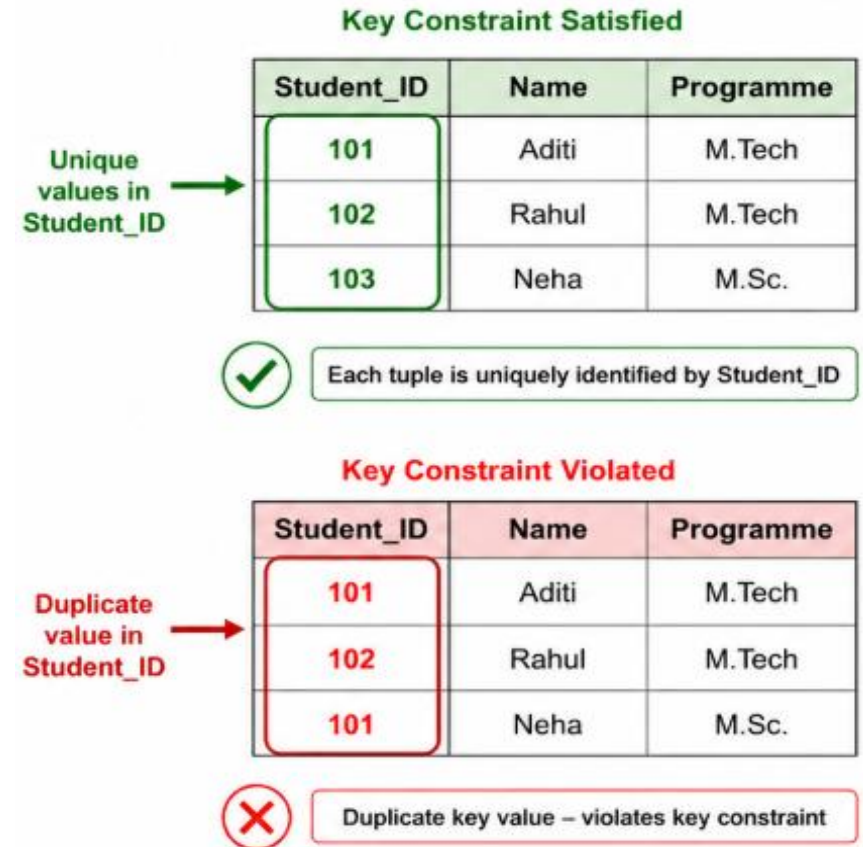
- A **domain constraint** specifies that the value of each attribute in a tuple must belong to its defined domain.
- A **domain** defines the set of permissible values for an attribute, including its **data type**, **range**, **format**, and other restrictions.
- Common domains include:
 - Integer
 - Real Number
 - Character/String
 - Boolean
 - Date
 - Time

Attribute	Domain	Valid Values
Age	Integer	18–60
CGPA	Decimal	0.0–10.0
Gender	CHAR(1)	M, F
Join_Date	DATE	Valid dates only

Key Constraints



- A **key** is one or more attributes that uniquely identify each tuple (row) in a relation.
- A **key constraint** ensures that no two tuples in a relation have the same key value.
- Every relation must contain at least one attribute or combination of attributes that uniquely identifies each tuple.
- Key constraints introduce the following types of keys:
 - Super Key
 - Candidate Key
 - Primary Key



Super Key



A Super Key is a set of one or more attributes that uniquely identifies each tuple (row) in a relation.

Characteristics

- Uniquely identifies every tuple in a relation.
- May contain **additional (redundant) attributes**.
- A relation can have **multiple super keys**.

Student_ID	Email	Name	Programme
101	aditi@bits.edu	Aditi	M.Tech
102	rahul@bits.edu	Rahul	M.Tech

Some of the Possible Super Keys:

- {Student_ID}
- {Email}
- {Student_ID, Name}
- {Student_ID, Programme}
- {Email, Name}

Candidate Key



A Candidate Key is a minimal Super Key that uniquely identifies each tuple in a relation.

Characteristics

- Uniquely identifies each tuple in a relation.
- Contains **no redundant attributes**.
- Every relation has **at least one Candidate Key**.
- One of the Candidate Keys is selected as the **Primary Key**.

Student_ID	Email	Name	Programme
101	aditi@bits.edu	Aditi	M.Tech
102	rahul@bits.edu	Rahul	M.Tech

Candidate Keys:

- {Student_ID}
- {Email}

Primary Key



A Primary Key is the Candidate Key selected to uniquely identify each tuple (row) in a relation.

Characteristics

- Chosen from the set of Candidate Keys.
- Must contain **unique** values.
- **Cannot contain NULL values.**
- Only **one Primary Key** can be defined for a relation.

Student_ID	Email	Name	Programme
101	aditi@bits.edu	Aditi	M.Tech
102	rahul@bits.edu	Rahul	M.Tech

Candidate Keys:

- {Student_ID}
- {Email}

Primary Key:

- {Student_ID}

Entity Integrity Constraint



- The Entity Integrity Constraint states that the **Primary Key** of a relation cannot contain **NULL** values.
- Since the **Primary Key** uniquely identifies each tuple (row) in a relation, every record must have a valid Primary Key value.
- Allowing **NULL** values in the Primary Key would make it impossible to uniquely identify a tuple.

✓ Valid Relation

Student_ID (Primary Key)	Name
101	Aditi
102	Rahul
103	Neha

No NULL values in Primary Key

Entity Integrity Satisfied

✗ Invalid Relation

Student_ID (Primary Key)	Name
101	Aditi
NULL	Rahul
103	Neha

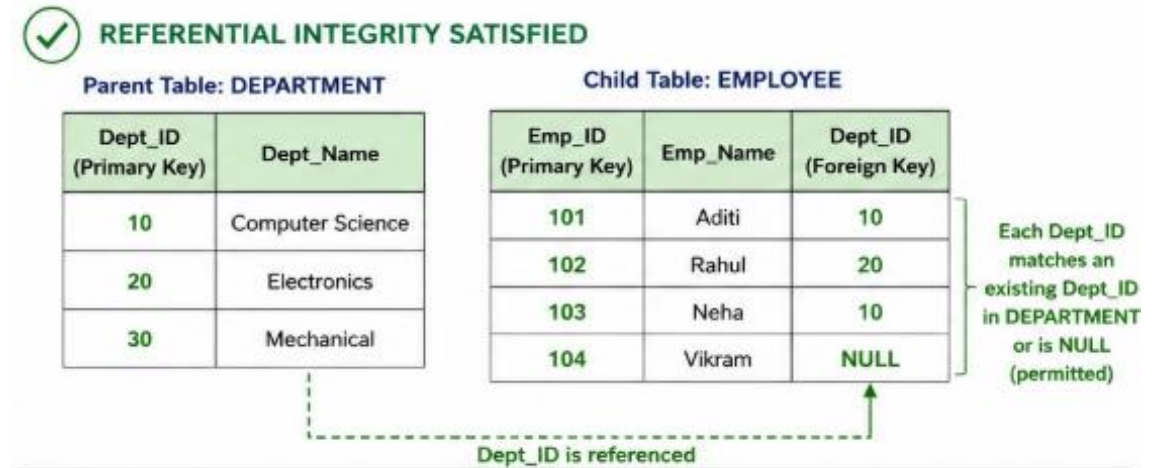
NULL value in Primary Key

Entity Integrity Violated

Referential Integrity Constraint



- Ensures that every **foreign key value** in a relation either:
 - matches an existing **primary key** value in the referenced relation, or
 - is **NULL** (if permitted).
- Prevents the creation of **orphan records** and maintains consistency between related tables.
- Ensures that relationships between tables remain **valid and consistent**



Other Types of Constraints



- Salary of employee should not exceed the salary of Employees supervisor.
- Maximum number of hours an employee can work on a project per week is 56 hours.
- Salary of employee should increase only.
- These constraints can be enforced through application programs.

Before creating database tables, we first need to understand the real-world objects, their properties, and the relationships among them. This process is called Data Modelling.

Why Data Modelling?

- Represents real-world requirements in a structured manner.
- Simplifies database design.
- Reduces redundancy and improves consistency.
- Serves as the foundation for database implementation.

Entity-Relationship (ER) Modelling



Entity-Relationship (ER) Modelling is a conceptual data modelling technique used to represent the structure of a database before implementation.

ER Model Represents

- **Entities** – Real-world objects about which data is stored.
- **Attributes** – Properties that describe an entity.
- **Relationships** – Associations between entities.

Why ER Modelling?

- Provides a visual representation of the database.
- Simplifies database design and communication.
- Helps identify entities, attributes, and relationships.
- Serves as the blueprint for designing relational databases.

Entity-Relationship (ER) Model

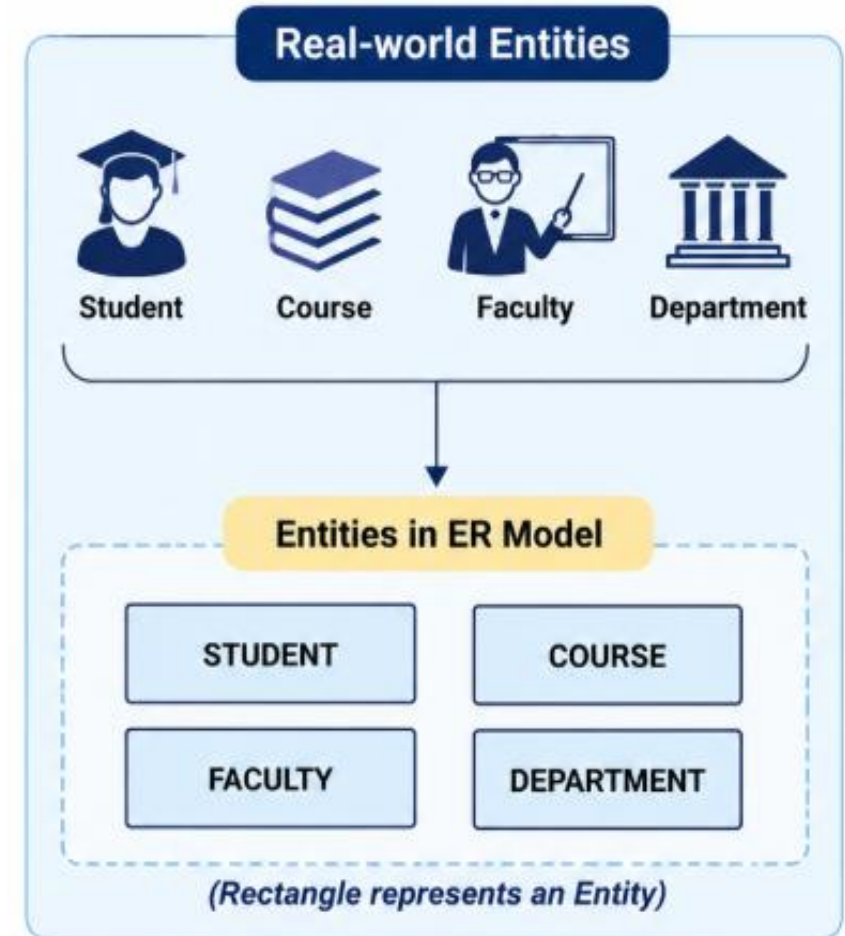


The Entity-Relationship (ER) Model is a conceptual data modelling technique used to represent the data requirements of an organization before database implementation.

The ER Model is represented using an Entity-Relationship Diagram (ER Diagram), which graphically depicts:

- **Entities** – Real-world objects about which data is stored.
- **Attributes** – Properties that describe an entity.
- **Relationships** – Associations among entities.

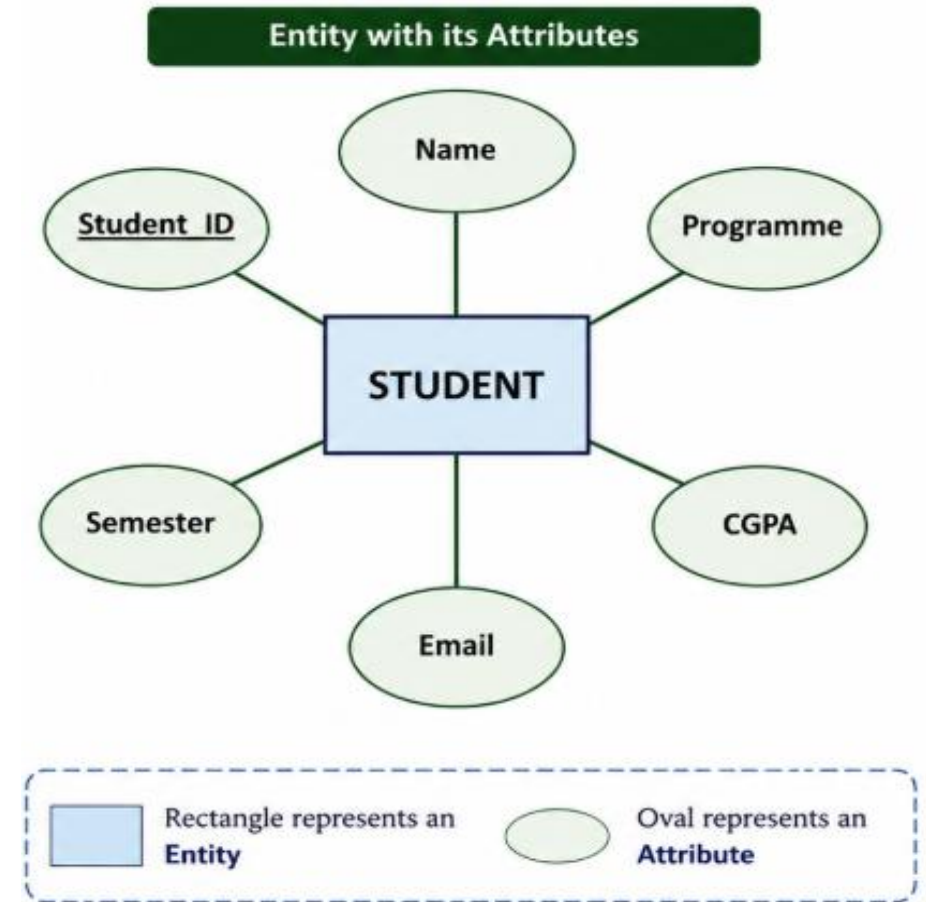
- An entity is a real-world object, person, place, event, or concept that can be uniquely identified and about which data is stored. Each entity represents a distinguishable object in the real world.
- Examples of entities include:
 - ❖ Student
 - ❖ Course
 - ❖ Faculty
 - ❖ Department
 - ❖ Employee



- **Attributes are the properties or characteristics that describe an entity.**
- Each entity is described by one or more attributes.
- Every entity instance has a value for each of its attributes.
- **Example**
- **Entity:** Student

Attributes:

- Student_ID
- Name
- Programme
- Semester
- CGPA
- Email



Based on their characteristics, attributes in an ER model can be classified into the following types:

- **Simple Attribute** – Cannot be divided into smaller components.
- **Composite Attribute** – Can be divided into smaller meaningful components.
- **Single-Valued Attribute** – Holds only one value for an entity.
- **Multi-Valued Attribute** – Can hold multiple values for an entity.
- **Derived Attribute** – Value is computed from one or more other attributes.

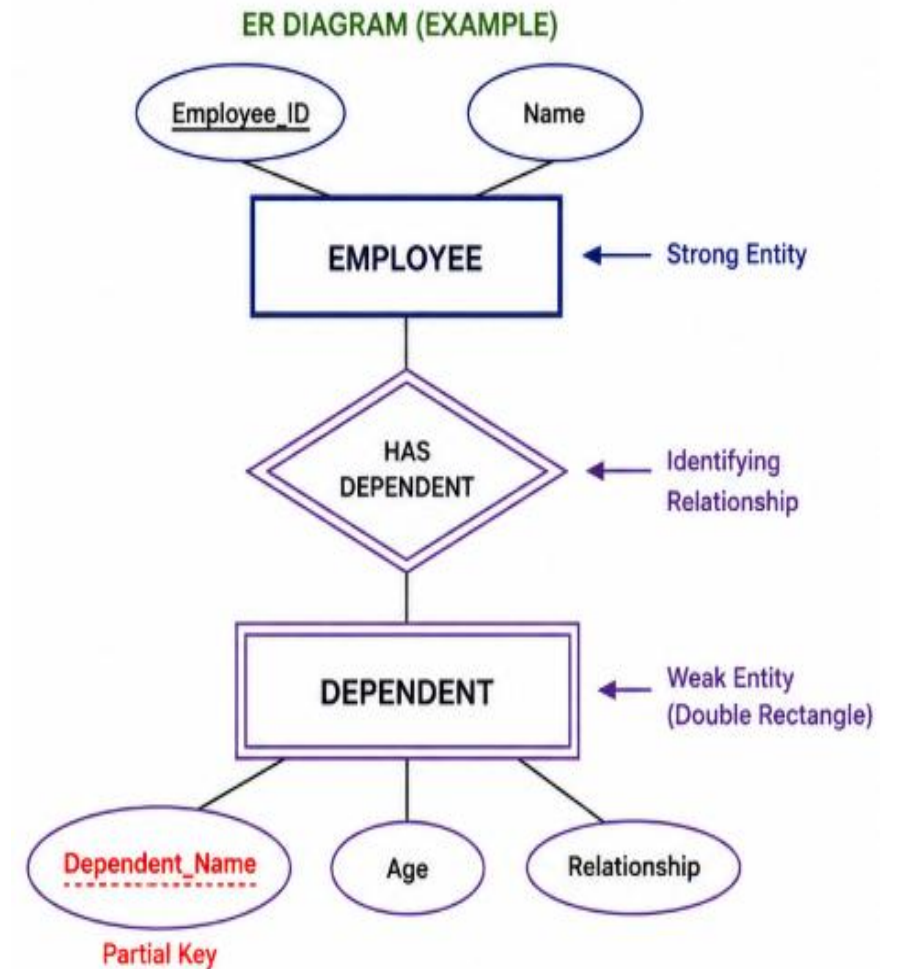
Weak Entity



- A **Weak Entity** is an entity that **cannot be uniquely identified by its own attributes**.
- Its existence depends on another entity, called the **Owner (or Identifying) Entity**.
- A Weak Entity is identified using:
 - The **Primary Key** of the Owner Entity, and
 - Its own **Partial Key**.

Example

- **Employee → Dependent**
- An employee can exist independently.
- A dependent cannot exist without an employee.



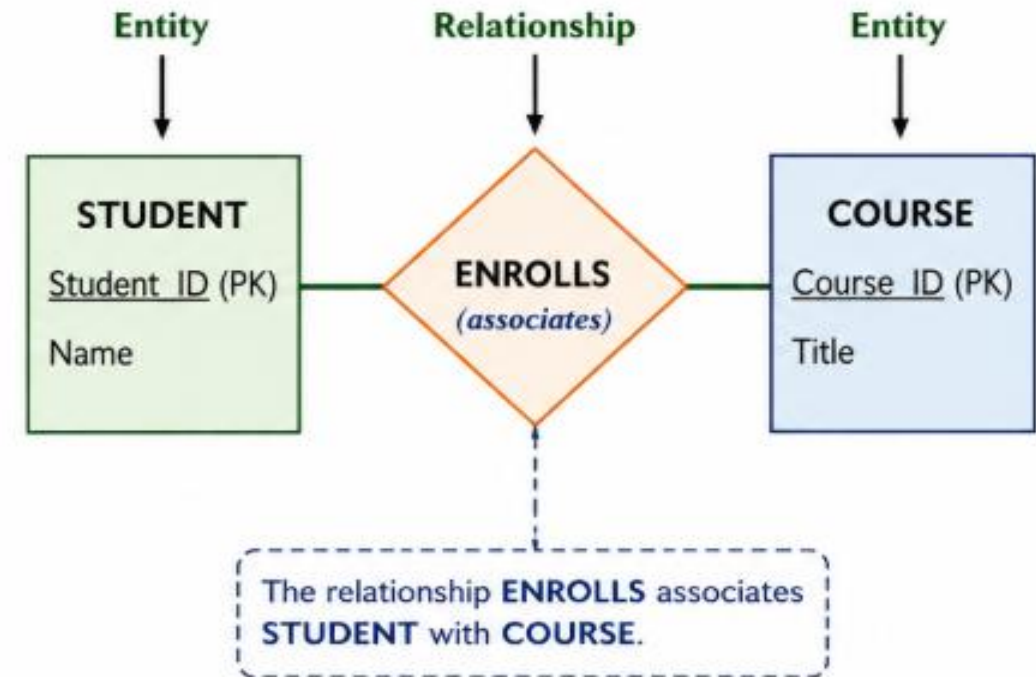
Relationship



- A relationship represents an association between two or more entities.
- It describes how entities are connected and interact with each other.

Examples

- A Student enrolls in a Course.
- A Faculty teaches a Course.
- A Department offers a Programme.



Notations Used in ER diagrams

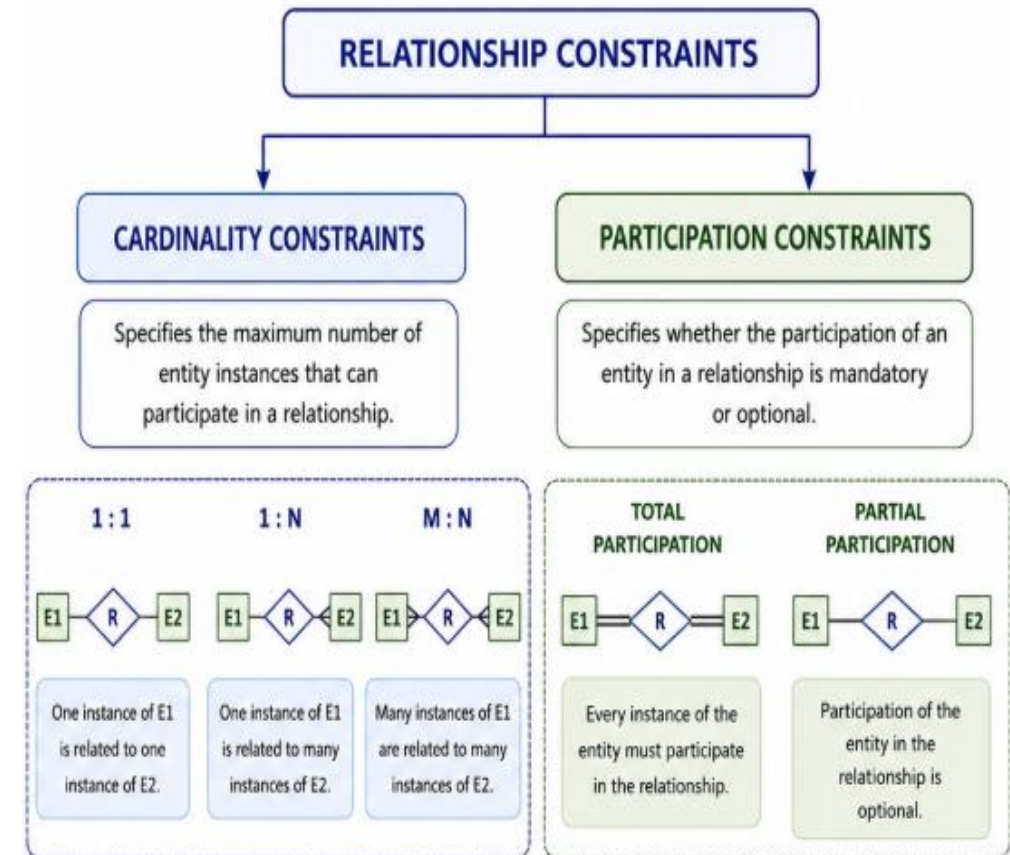


S. No.	Symbol	Notation Name	Meaning / Usage
1.		Strong Entity Type	Represents a strong (independent) entity in the real world.
2.		Weak Entity Type	Represents a weak entity that depends on another entity for existence.
3.		Relationship Type	Represents an association between two or more entity types.
4.		Identifying Relationship Type	Represents a relationship that identifies a weak entity with its owner.
5.		Attribute	Represents an attribute (property) of an entity or relationship.
6.		Key Attribute	Represents the primary key attribute that uniquely identifies an entity.
7.		Multivalued Attribute	Represents an attribute that can have multiple values for an entity.
8.		Derived Attribute	Represents an attribute whose value is derived from another attribute.
9.		Partial Key	Represents the partial key of a weak entity (identified together with owner's key).
10.		Composite Attribute	Represents an attribute composed of smaller sub-attributes.

Relationship Constraints



- Relationship constraints specify how entities participate in a relationship and define the nature of associations between entities.
- Two types of relationship constraints:
 - **Cardinality Constraints**
 - ✓ Specify the **maximum number** of entity instances that can participate in a relationship.
 - ✓ Examples: **1:1**, **1:N**, **M:N**
 - **Participation Constraints**
 - ✓ Specify whether the participation of an entity in a relationship is **mandatory or optional**.
 - ✓ Types: **Total Participation** and **Partial Participation**



Exercise



- The COMPANY database keeps track of a company's employees, departments, and projects. Suppose that after the requirements collection and analysis phase, the database designers provide the following description of the miniworld—the part of the company that will be represented in the database.
- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.

- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- We store each employee's name, Social Security number, address, salary, gender, and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee (who is another employee).
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, gender, birth date, and relationship to the employee.

Identify the Entities



1. Employee
2. Department
3. Project
4. Dependent

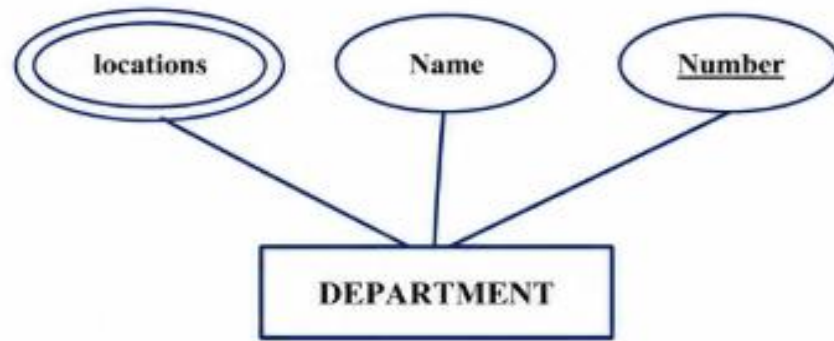
Contd..

innovate

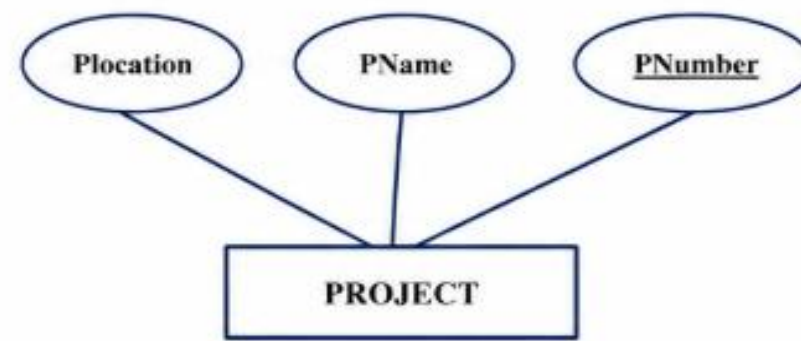
achieve

lead

DEPARTMENT



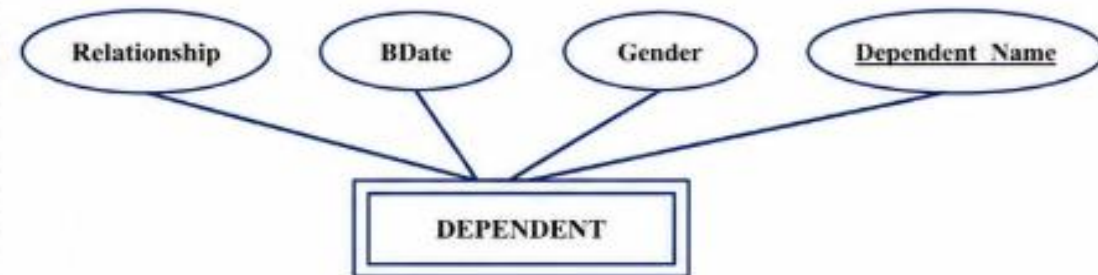
PROJECT



EMPLOYEE



DEPENDENT

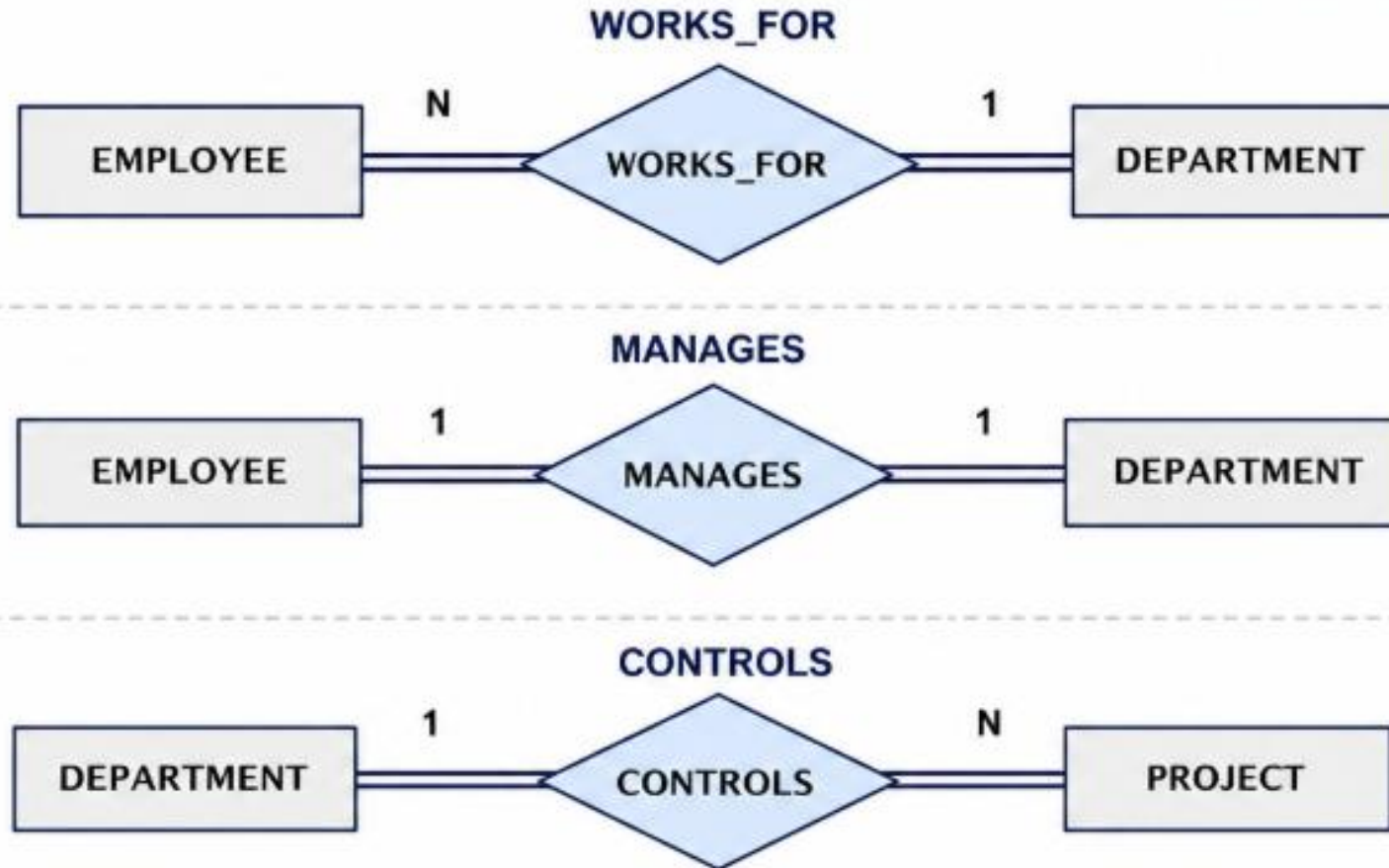


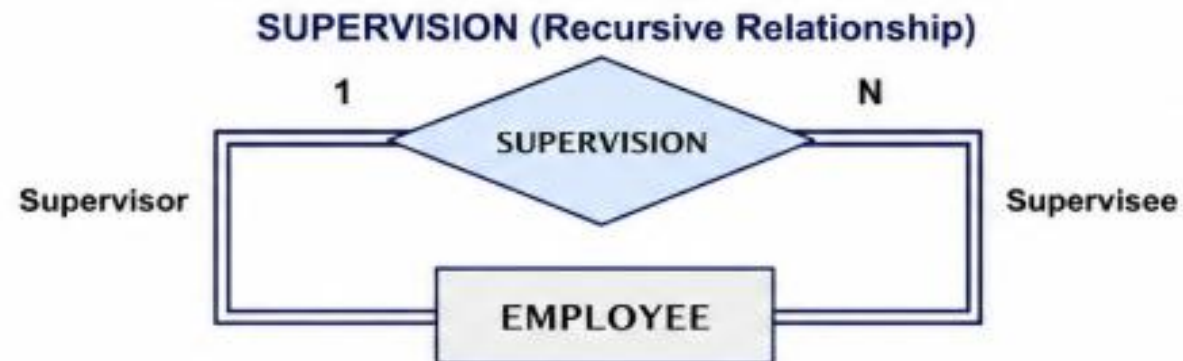
Relationships



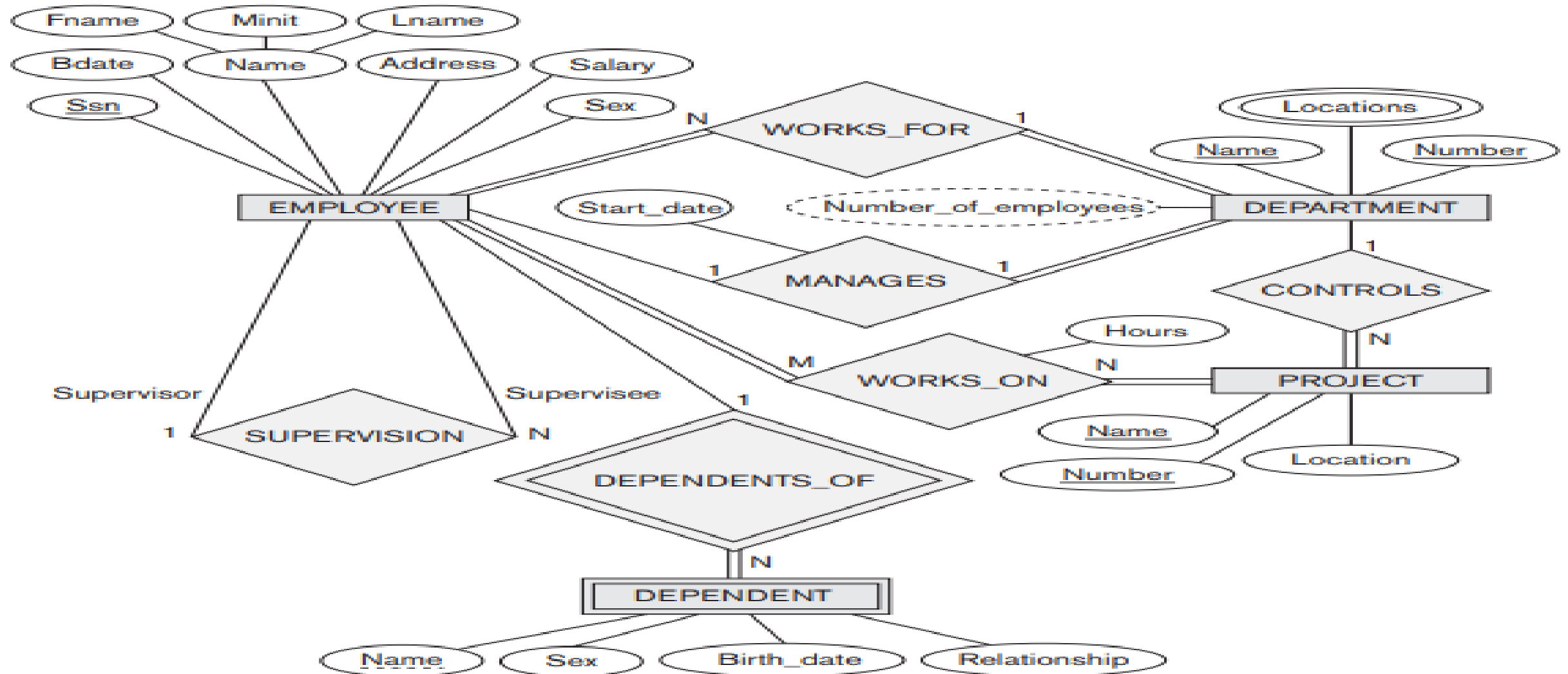
1. Employee manages Department
2. Employee works for Department
3. Department controls project
4. Employee supervises Employee
5. Employee works on projects
6. Employee has Dependents

Contd..





Complete ER Diagram



- After an ER model is converted into relational tables, the resulting tables may contain redundant data.
- Redundancy can lead to several problems during data insertion, updation, and deletion.
- **Normalization** is the process of organizing data into well-structured relations to:
 - ✓ Reduce data redundancy
 - ✓ Improve data consistency
 - ✓ Eliminate modification anomalies
 - ✓ Improve data integrity

Why is Normalization Required?



- Suppose the COMPANY database is stored in a single relation as shown

EmpSSN	EmpName	DeptNo	DeptName	DeptLocation	MgrSSN
123456789	John Smith	5	Research	Houston	333445555
453453453	Joyce English	5	Research	Houston	333445555
666884444	Ramesh Narayan	4	Administration	Stafford	987654321
987987987	Ahmad Jabbar	4	Administration	Stafford	987654321

- The department information is repeated for every employee belonging to the same department.
- This redundancy increases storage requirements and may lead to:
 - ☐ Update Anomalies
 - ☐ Insertion Anomalies
 - ☐ Deletion Anomalies

Normal Forms



- Normalization is achieved by transforming a relation into a sequence of well-structured relations known as **Normal Forms**.
- Each normal form addresses a specific type of redundancy and dependency.

Normal Form	Objective
First Normal Form (1NF)	Eliminate repeating groups and ensure atomic attribute values.
Second Normal Form (2NF)	Eliminate partial functional dependencies.
Third Normal Form (3NF)	Eliminate transitive dependencies.

First Normal Form



A relation is in **First Normal Form (1NF)** if:

- Every attribute contains **atomic (indivisible) values**.
- There are **no repeating groups or multivalued attributes**.
- Each row is uniquely identifiable using a primary key.

Converting a relation to 1NF involves ensuring that each cell contains only a single value.

Before 1NF (Not in 1NF)

EmpID	Employee Name	Phone Numbers
E101	John Smith	9876543210, 9988776655
E102	Joyce English	9123456789
E103	Ramesh Narayan	9012345678, 9090909090

↓

After 1NF (In 1NF)

EmpID	Employee Name	Phone Number
E101	John Smith	9876543210
E101	John Smith	9988776655
E102	Joyce English	9123456789
E103	Ramesh Narayan	9012345678
E103	Ramesh Narayan	9090909090

Second Normal Form

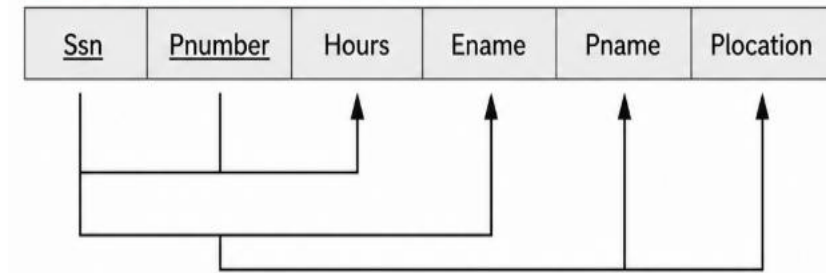


A relation is in **Second Normal Form (2NF)** if:

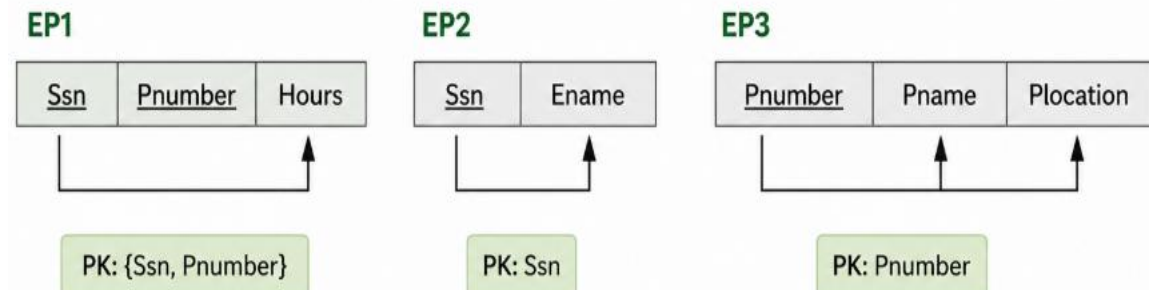
- It is already in **First Normal Form (1NF)**.
- Every **non-prime attribute** is **fully functionally dependent** on the **entire candidate key**.
- There are **no partial functional dependencies**.

2NF is relevant only when a candidate key is composite (consists of more than one attribute).

EMP_PROJ (in 1NF but not in 2NF)



↓
2NF Normalization



Third Normal Form

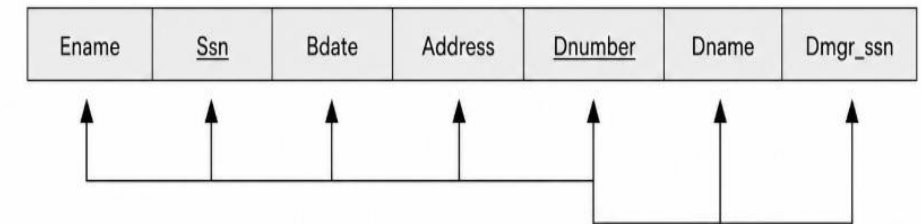


A relation is in **Third Normal Form (3NF)** if:

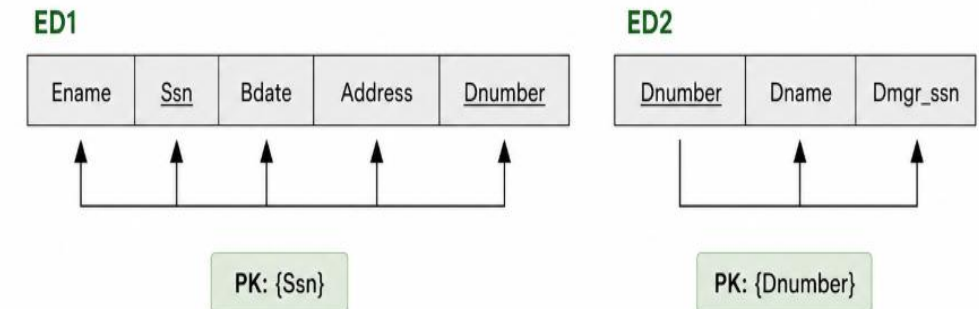
- It is already in **Second Normal Form (2NF)**.
- There are **no transitive functional dependencies**.
- Every **non-prime attribute** depends **only on a candidate key**, and not on another non-prime attribute.

A **transitive dependency** occurs when a non-prime attribute depends on another non-prime attribute instead of depending directly on a candidate key.

EMP_DEPT (in 2NF but not in 3NF)



3NF Normalization





Thank You

